Software Test Plan (STP) - File Compression Tool (Basic ZIP Implementation)

Project: File Compression Tool (Basic ZIP Implementation)
Version: 1.0
Authors: Debaditya Chakrabarti, Divisha Gupta, Greeshma N, Chandrakant V G
Date: 02-10-2026
Status: Draft – Part 1

1. Introduction

This test plan follows the supplied PES University Software Test Plan structure. Requirement IDs are taken from the project SRS and are used consistently for traceability.

Purpose: This document defines the verification and validation approach for the Huffman-based File Compression Tool v1.0. It covers objectives, scope, strategy, environment, security validation, responsibilities, test cases, traceability, metrics, and acceptance criteria.
Scope: Testing covers command parsing, input validation, Huffman frequency analysis/tree construction, encoding/decoding, archive creation/parsing, metadata reporting, integrity validation, safe output handling, performance, and error handling. Third-party ZIP compatibility, GUI behavior, network services, and operating-system internals are outside the current scope.
References: File Compression Tool SRS v1.0, Software Architecture and Design Specification v1.0, Group 14 project statement, IEEE 29148-2018.
Definitions: SRS (Software Requirements Specification), STP (Software Test Plan), RTM (Requirement Traceability Matrix), CLI (Command-Line Interface), TC (Test Case).

2. Test Items

- CLI command and argument validation
- Compression Controller
- Huffman frequency table and tree construction
- Huffman encoding and decoding
- Project-defined archive creation and parsing
- Metadata inspection/reporting
- Integrity and checksum validation
- Output path validation and overwrite protection
- Error handling and exit status
- Performance of compression/decompression

3. Features to be Tested

Coverage baseline: all 14 functional requirements (FR-CMP-01..07, FR-DEC-01..04, FR-INFO-01, FR-ERR-01, FR-OUT-01), the performance/reliability/usability/safety requirements, and the four security requirements are covered by test cases or review/build verification.

Features mapped to SRS requirement IDs:
- FR-CMP-01: Accept valid input/output paths
- FR-CMP-02: Calculate byte frequencies
- FR-CMP-03: Construct Huffman tree
- FR-CMP-04: Encode bytes into a bit stream
- FR-CMP-05: Store archive metadata required for decompression
- FR-CMP-06: Report compression statistics
- FR-CMP-07: Restore the exact original byte sequence
- NFR-PERF-01: Reference 10 MB file operation target ≤ 5 s
- SEC-REQ-01: Reject malformed/tampered archives
- SEC-REQ-02: Prevent unsafe output paths and path traversal

4. Features Not to be Tested

- Commercial ZIP/DEFLATE interoperability
- GUI functionality (not part of the current CLI scope)
- Network/cloud storage functionality
- Compiler/runtime internals outside the application boundary
- Third-party compression libraries replacing the required Huffman implementation

5. Test Approach / Strategy

Levels:
- Unit tests: Huffman codec, bit I/O, archive parsing, validation and utility functions.
- Integration tests: Controller ↔ codec ↔ archive manager ↔ file I/O.
- System tests: end-to-end compression/decompression from CLI.
- Acceptance tests: representative user scenarios and project requirements.

Types:
- Functional testing for all SRS functional requirements.
- Negative/error testing for invalid paths, malformed archives, corrupt metadata, and permission failures.
- Regression testing after defect fixes.
- Performance testing for file size, operation time, and memory-sensitive scenarios.
- Security testing for tampering, path traversal, output-size abuse, and information leakage.

Entry Criteria: stable build available; compiler/toolchain working; reference files prepared; expected outputs and test environment ready.
Exit Criteria: all planned test cases executed; all critical/high defects resolved or formally

accepted; all SRS security requirements tested; requirement coverage documented; acceptance criteria met.

5.1 Security Validation

Security Validation Objectives:
1. Verify that malformed or tampered archives cannot be treated as trusted input.
2. Verify that archive metadata cannot cause unsafe file-system access or accidental overwrite.

Security validation activities:
- Check archive magic/version and reject unsupported formats.
- Modify header, tree metadata, payload bits, and checksum; verify rejection or integrity failure.
- Attempt `../`, absolute, and platform-specific unsafe output paths; verify sanitization/rejection.
- Attempt declared output-size overflow; verify decoding stops safely.
- Verify diagnostic messages do not expose full file contents or sensitive paths beyond what is necessary.
- Run negative/fuzz-style malformed-input tests on command arguments and archive bytes.
- Confirm non-zero exit status on security validation failure.

Security acceptance criteria: tampered/malformed archives are rejected; checksum/integrity failures do not produce trusted output; absolute paths and parent-directory traversal are rejected; decoded output cannot exceed the declared original size; complete input-file contents do not appear in diagnostics; and security failures return a non-zero exit status.

6. Test Environment

Hardware: Standard development laptop/desktop; local storage.
Software: C++17-or-later compiler (GCC/Clang/MSVC), operating system file system, project executable, and reference test files.
Tools: command shell, Git, debugger, Graphviz/PlantUML for design, and a unit-test framework if used by the team.
Test Data: empty file, single-byte file, repeated-byte file, text file, binary file, random data, approximately 10 MB reference file, malformed archives, tampered archives, and unsafe path metadata.

7. Test Schedule

Milestones:
- Requirements/design review: 02-Oct-2026
- Test case design and data preparation: 03-Oct-2026
- Unit/integration test execution: 04-Oct-2026 to 05-Oct-2026
- System/security/performance testing: 06-Oct-2026
- Final regression and test summary: 07-Oct-2026

8. Test Deliverables

- Software Test Plan (this document)
- Test Case Set and Test Data
- Test Scripts/commands where applicable
- Test Execution Logs
- Defect Reports
- Requirements Traceability Matrix
- Security Validation Evidence
- Final Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
QA / Test Lead	Chandrakant V G	Own test plan, security validation, execution coordination, defect triage
Test Engineer	Debaditya Chakrabarti	Design functional/integration cases and archive/file I/O tests
Developer / Algorithm Lead	Divisha Gupta	Support unit tests for Huffman tree, encoding and decoding; fix algorithm defects
Architecture / Integration Lead	Greeshma N	Maintain SRS/SAD traceability, integration testing, performance and release readiness

10. Risks and Mitigation

Risk	Mitigation
Huffman bit-boundary defect	Use single-byte/repeated-byte/round-trip tests and dedicated bit I/O unit tests.
Malformed archive causes unsafe output	Fail-closed validation, checksum/size checks, path sanitization and security tests.
Large file performance misses target	Use buffered I/O, benchmark early, profile hotspots and optimize only after measurement.

11. Assumptions & Dependencies

- A working C/C++ compiler and build configuration will be available.
- Reference files and expected checksums will be prepared before system testing.
- The team-defined archive format remains stable after the design baseline.
- Tests are performed with sufficient file-system permissions.
- No network or external service is required.

12. Suspension & Resumption Criteria

Suspend testing if:
- The executable cannot start or blocks more than 30% of planned cases.
- The archive format changes without an updated build/design baseline.
- Test environment or file-system access is unavailable for more than 4 hours.

Resume testing if:
- Blocking defects are resolved and a stable build is delivered.
- Test data/environment is restored.
- The SRS/SAD baseline is updated when a design change caused the suspension.

13. Test Case Management & Traceability

RTM ensures mapping of SRS requirements to test cases and system tests.
Examples:
- FR-CMP-01 → TC-01 → ST-01
- FR-CMP-02 → TC-02 → ST-02
- FR-CMP-03 → TC-03 → ST-03
- FR-CMP-04 → TC-04 → ST-04
- FR-CMP-05 → TC-05 → ST-05
- FR-CMP-06 → TC-06 → ST-06
- FR-CMP-07 → TC-07 → ST-07
- SEC-REQ-01 → TC-08 → ST-08
- SEC-REQ-02 → TC-09 → ST-09
- NFR-PERF-01 → TC-10 → ST-10

13.1 Test Cases

TC ID	Requirement	Feature	Precondition / Input	Expected Result	Pass Criterion	System Test
TC-01	FR-CMP-01	Valid input/output paths	Run compress on a readable small text file	Archive created; exit 0; success message and output path shown	Archive exists and is readable	ST-01
TC-02	FR-CMP-02	Frequency analysis	Compress a file containing repeated and distinct bytes	Frequency table/tree reflects all byte values present	Expected symbol set is represented	ST-02
TC-03	FR-CMP-03	Huffman tree construction	Compress a known small input with known frequency distribution	Tree/code generation completes and produces prefix-free codes	No duplicate/conflicting codes; operation succeeds	ST-03
TC-04	FR-CMP-04	Bitstream encoding/decoding	Compress and decompress a file whose encoded stream ends mid-byte	Decoded output matches original byte-for-byte	Hash/checksum of source and restored file equal	ST-04
TC-05	FR-CMP-05	Archive metadata	Run info on a generated archive	Magic/version, filename, sizes, tree metadata	Metadata matches source and payload	ST-05

				and checksum are readable		
TC-06	FR-CMP-06	Compression statistics	Compress text and binary reference files	Tool reports original size, compressed size and ratio without division errors	Statistics are numerically consistent	ST-06
TC-07	FR-CMP-07	Lossless round trip	Compress then decompress empty, 1-byte, text and binary files	Restored files are byte-for-byte identical to originals	All reference file hashes match	ST-07
TC-08	SEC-REQ-01	Tampered archive rejection	Modify header/payload/checksum bytes of a valid archive	Decompressor rejects invalid archive or reports integrity failure and produces no trusted output	Non-zero exit; no misleading success	ST-08
TC-09	SEC-REQ-02	Unsafe path protection	Provide archive metadata containing ../ or absolute output path	Decompressor rejects/sanitizes unsafe path and does not write outside target directory	No path traversal and no unintended overwrite	ST-09
TC-10	NFR-PERF-01	Performance	Compress and decompress a 10 MB local reference file	Each operation completes within 5 seconds on the reference machine, subject to documented hardware	Timing recorded; correctness also verified	ST-10

The following 10 test cases provide the minimum Part-1 coverage requested for functional, non-functional, negative, and security requirements.

13.2 Security and Requirement Traceability

Traceability completeness: every SRS functional requirement has at least one test case; security objectives are validated by TC-08/TC-09; NFR-PERF-01 is validated by TC-10; safety/output requirements are validated by TC-09; and maintainability/portability/testability are additionally checked during design/build/unit-test reviews.

Requirement mapping summary: FR-CMP-01→TC-01; FR-CMP-02→TC-02; FR-CMP-03→TC-03; FR-CMP-04→TC-04; FR-CMP-05→TC-05; FR-CMP-06→TC-06; FR-CMP-07/FR-DEC-02→TC-07; FR-DEC-01/FR-DEC-03/FR-ERR-01/NFR-REL-01/NFR-SEC-01/NFR-SEC-03/NFR-SEC-04→TC-08; FR-DEC-04/FR-OUT-01/NFR-SAFE-01/NFR-SEC-02→TC-09; NFR-PERF-01→TC-10; FR-INFO-01/NFR-USE-01→TC-05/TC-01.

SEC-OBJ-01 / SEC-REQ-01 → TC-08 → archive tampering, malformed header, checksum validation.

SEC-OBJ-02 / SEC-REQ-02 → TC-09 → path traversal and unsafe output validation.

SAF-01 → TC-09 → output overwrite protection.

SAF-02 → TC-01/TC-08/TC-10 → failure handling and safe output creation.
NFR-PERF-01 → TC-10 → measured compression/decompression time.
FR-CMP-07 → TC-07 → byte-for-byte round-trip verification.

14. Test Metrics & Reporting

Metrics collected:
- Test cases executed / planned
- Pass / fail percentage
- Requirement coverage percentage
- Security requirement coverage
- Defect count, severity, and aging
- Mean time to resolve critical/high defects
- Compression/decompression execution time
- Compression ratio on reference files

Reports:
- Daily execution status
- Defect summary
- Security validation summary
- Final Test Summary Report

15. Approvals

Role	Name	Signature / Date
QA / Test Lead	Chandrakant V G	Pending / 02-10-2026
Test Engineer	Debaditya Chakrabarti	Pending / 02-10-2026
Developer / Algorithm Lead	Divisha Gupta	Pending / 02-10-2026
Architecture / Integration Lead	Greeshma N	Pending / 02-10-2026